

A Project Report

On

Multimodal media search: Semantic image embeddings retrieval, VLM & Face recognition

Submitted by

Aryan Nilesh Marthak

Enrolment No.: **220170111007**

Team ID: **985275**

ELECTRONICS & COMMUNICATION ENGINEERING, Semester VIII

Month & Year: April 2026

Prepared as part of the requirement for the subject of

Subject Name: Internship / Project

Subject Code: 3181101

Under the Guidance of

Prof. Jagruti K. Naik

Assistant Professor

Dept. of Electronics &

Communication Engineering

Dr. Arun Nandurbarkar

Professor and Head

Dept. of Electronics &

Communication Engineering



**Vishwakarma Government Engineering
College**

(Institute Code: 017)

Chandkheda, Ahmedabad

Affiliated with

Gujarat Technological University



Vishwakarma Government Engineering College

(Institute Code: 017)

Chandkheda, Ahmedabad

Affiliated to Gujarat Technological University, Ahmedabad



CERTIFICATE

This is to certify that the Project report entitled “**Multimodal media search: Semantic image embeddings retrieval, VLM & Face recognition**” has been carried out by **Aryan Nilesh Marthak** bearing Enrolment Number **220170111007** under my guidance in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering in Electronics & Communication Engineering**, Semester VIII of Gujarat Technological University, Ahmedabad during the academic year **2025–2026**.

The work embodied in this report is original and has been carried out by the student in the Department of Electronics & Communication Engineering, Vishwakarma Government Engineering College, Chandkheda, Ahmedabad.

Internal Guide

Prof. Jagruti K. Naik
Assistant Professor
Department of Electronics &
Communication Engineering

Head of Department

Dr. Arun Nandurbarkar
Professor and Head
Department of Electronics &
Communication Engineering

DECLARATION

I hereby declare that the Project report titled “**Multimodal media search: Semantic image embeddings retrieval, VLM & Face recognition**” submitted by me to Gujarat Technological University, Ahmedabad in partial fulfilment of the requirement for the award of the degree of *Bachelor of Engineering in Electronics & Communication Engineering* is a record of bona fide work carried out by me under the supervision of **Prof. Jagruti K. Naik**, Assistant Professor, Department of Electronics & Communication Engineering, Vishwakarma Government Engineering College, Chandkheda, Ahmedabad.

I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this or any other institute or university. All sources of information and help received have been duly acknowledged.

Aryan Nilesch Marthak

Enrolment No.: **220170111007**

Date: _____

Place: Ahmedabad

ACKNOWLEDGEMENTS

The successful completion of this project would not have been possible without the guidance, encouragement, and assistance of several individuals whose contributions deserve sincere acknowledgement.

First and foremost, I express my deepest gratitude to my internal guide, **Prof. Jagruti K. Naik**, Assistant Professor, Department of Electronics & Communication Engineering, Vishwakarma Government Engineering College, for her continuous support, valuable suggestions, and technical mentorship throughout the development of this project. Her constant encouragement, patience, and constructive feedback have been instrumental in shaping both the technical direction and the academic quality of this work.

I extend my heartfelt thanks to **Dr. Arun Nandurbarkar**, Head of the Department of Electronics & Communication Engineering, and to the Principal of Vishwakarma Government Engineering College, Chandkheda, Ahmedabad, for providing an excellent academic environment and the infrastructure required to carry out this project. I am also grateful to the entire faculty of the Department of Electronics & Communication Engineering for the knowledge and training imparted during my undergraduate studies, which formed the foundation on which this project was built.

I acknowledge the open-source communities behind *OpenAI CLIP*, *HuggingFace Transformers*, *PyTorch*, *FastAPI*, *Qdrant*, *React*, *DeepFace*, and *PostgreSQL*, without whose tooling and documentation this project would not have been feasible. I am also grateful to the contributors of the *open_clip* library and the *SmolVLM* model by HuggingFace, which form the core of the AI subsystem.

Finally, I express my heartfelt appreciation to my family and friends for their patience, understanding, and unwavering moral support during the course of this project.

Aryan Nilesh Marthak
Enrolment No.: **220170111007**

ABSTRACT

The exponential growth of personal and professional digital image collections has created a widespread challenge: traditional filename-based and date-based retrieval methods are wholly inadequate for content-based queries. A user who wishes to find “a photo of my dog at the beach” or “the birthday cake from last year” cannot rely on folder hierarchies or manually assigned tags, particularly when image libraries contain tens of thousands of photographs accumulated over years of smartphone use.

This report presents **Media-Search**, a self-hosted, AI-powered image retrieval and face recognition platform designed to bridge the gap between natural language and visual content. The system combines four distinct AI components: a CLIP (ViT-L/14) model for semantic vision-language embedding, a SmolVLM vision-language model for generating rich image descriptions, DeepFace for accurate face detection and embedding, and DBSCAN clustering for grouping detected faces into person identities. Together, these components enable two complementary search modes: a fast two-stage CLIP recall and local re-ranking pipeline, and a deep hybrid BM25-plus-CLIP search over VLM-generated image descriptions, as well as the ability to retrieve all images containing a specific named individual.

The platform is built as a modern full-stack web application, comprising a FastAPI backend with asynchronous SQLAlchemy and PostgreSQL for metadata persistence, Qdrant as the vector similarity database for CLIP embeddings, Redis for embedding caching, and a React 18 single-page frontend. The system supports multi-user operation with JWT-based authentication, strict per-user data isolation, image sharing with configurable view and download permissions, and real-time processing status updates delivered via Server-Sent Events (SSE).

Search performance on a workstation leveraging an NVIDIA RTX 3050 GPU demonstrates typical query latency of 50 to 150 ms for normal CLIP search and 100 to 300 ms for deep BM25-CLIP hybrid search when VLM descriptions have been pre-generated. The face detection and clustering pipeline operates as a background task, processing uploaded images asynchronously and updating cluster assignments automatically as new faces are encountered.

This report documents the problem background, related literature, system analysis, design, implementation details, results, and directions for future work.

Table of Contents

Institute Certificate	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
1.5 Motivation	4
1.6 Organisation of the Report	4
2 Literature Review	5
2.1 Overview	5
2.2 Vision-Language Pre-Training and CLIP	5
2.3 Two-Stage Retrieval and Local Re-Ranking	6
2.4 Vision-Language Models for Image Captioning	6
2.5 Face Detection and Recognition	7
2.6 Unsupervised Face Clustering with DBSCAN	7
2.7 Vector Databases for Approximate Nearest-Neighbour Search	8
2.8 Hybrid Keyword-Semantic Search	8
2.9 Summary of Insights	9
3 System Analysis	10
3.1 Study of the Existing System	10
3.1.1 Limitations of Existing Systems	10
3.2 Proposed System	11
3.3 Functional Requirements	11
3.3.1 Authentication and User Management	11

3.3.2	Image Upload and Processing	12
3.3.3	Semantic Search	12
3.3.4	Face Clustering and Search	13
3.4	Non-Functional Requirements	13
3.5	Technology Selection Rationale	14
3.6	Feasibility Study	14
3.6.1	Technical Feasibility	14
3.6.2	Operational Feasibility	15
3.6.3	Economic Feasibility	15
3.7	Project Timeline	16
4	System Design	17
4.1	High-Level Architecture	17
4.2	Search Pipeline Design	18
4.2.1	Mode 1 — Global CLIP Search	18
4.2.2	Mode 2 — CLIP Search with Local Re-Ranking	18
4.2.3	Mode 3 — Deep Hybrid Search (BM25 + CLIP)	18
4.3	Entity-Relationship Diagram	19
4.4	Data Flow Diagram	20
4.4.1	Level-0 DFD (Context Diagram)	20
4.4.2	Level-1 DFD	20
4.5	Use Case Diagram	21
4.6	Sequence Diagram — Semantic Search	21
4.7	Sequence Diagram — Image Upload and Processing	22
5	Implementation	23
5.1	Development Environment	23
5.2	Backend Implementation	23
5.2.1	Application Entry Point	23
5.2.2	JWT Authentication	24
5.2.3	CLIP Embedding Service	25
5.2.4	Local Re-Ranking	26
5.2.5	VLM Integration	26
5.2.6	Face Detection and Clustering Pipeline	27
5.2.7	Hybrid Deep Search	28
5.2.8	Server-Sent Events	28
5.3	Frontend Implementation	29
5.3.1	Technology Stack	29
5.3.2	Authentication Context	29

5.3.3	Image Upload Component	29
5.3.4	Design System	29
5.4	Docker Compose Deployment	30
6	Results and Discussion	31
6.1	Functional Verification	31
6.2	Search Quality Evaluation	32
6.2.1	Evaluation Metrics	32
6.2.2	Mode 1 — Global CLIP Search	34
6.2.3	Mode 2 — CLIP with Local Re-Ranking	34
6.2.4	Mode 3 — Deep Hybrid Search	34
6.2.5	Comparison with Existing Systems	35
6.3	Performance Observations	36
6.4	System Screenshots	36
6.5	Failure Analysis	40
7	Conclusion	42
7.1	Summary of Contributions	42
7.2	Limitations	42
7.3	Future Work	43
7.4	Concluding Remarks	43
	References	44
	List of Abbreviations	45

List of Figures

4.1	High-level system architecture of the AI-Powered Media Search system	17
4.2	Entity-Relationship Diagram of the Media-Search database schema . . .	19
4.3	Level-0 DFD (Context Diagram)	20
4.4	Level-1 DFD of the Media-Search system	20
4.5	Use Case Diagram for the Media-Search system	21
4.6	Sequence diagram for the semantic search flow	21
4.7	Sequence diagram for the image upload and background processing flow	22
6.1	Retrieval quality comparison between CLIP search with local re-ranking and the deep hybrid BM25+CLIP mode across 499 Flickr8k queries	33
6.2	Search response latency breakdown by operation and mode (Hardware accelerated, NVIDIA RTX 3050)	34
6.3	User registration form demonstrating the secure signup process	37
6.4	Main gallery interface displaying all uploaded media for the authenticated user	37
6.5	Detailed view of a selected image within the gallery	38
6.6	AI-generated natural language description of an image produced by the VLM	38
6.7	Additional example of automated attribute extraction and scene description	39
6.8	Face clusters view grouping detected individuals automatically	39
6.9	Detailed view of all images associated with a specific person	40
6.10	Search interface displaying the fallback state when no matching results are found	40

List of Tables

3.1	Comparison of Existing Image Search Systems	10
3.2	Non-Functional Requirements	13
3.3	Technology Choices and Rationale	14
3.4	Project Schedule (Gantt Summary)	16
6.1	Functional Requirement Verification Summary	31
6.2	Quantitative Comparison of Search Modes (CLIP: Full 8K Dataset, Deep Hybrid: 100-Image Subset)	33
6.3	Qualitative Comparison with Existing Image Retrieval Systems	35
6.4	Search Response Times (500 images, Hardware Accelerated, NVIDIA RTX 3050)	36

Chapter 1

Introduction

1.1 Background

The digitalisation of everyday life has produced an unprecedented volume of personal image data. The average smartphone user accumulates thousands of photographs each year, and professional photographers, content creators, and media organisations manage libraries that run into the hundreds of thousands of images. Cloud storage services have made storing these collections trivially inexpensive, but the problem of *finding* a specific image within a large collection has grown correspondingly harder.

Conventional image retrieval relies on metadata: filenames, directory paths, creation timestamps, and manually assigned tags. These approaches suffer from two fundamental limitations. First, the burden of labelling images accurately and consistently falls entirely on the user; in practice, most photographs are never tagged, and even when they are, the tags rarely capture the full semantic content of the image. Second, metadata-based retrieval requires the user to remember *metadata* rather than *content*. A query such as “that photo of the sunset over the mountains from the beach holiday” cannot be expressed naturally as a filename or date search.

Content-based image retrieval (CBIR) systems emerged in the 1990s to address these limitations by indexing images according to their visual content rather than their metadata. Early CBIR systems used hand-crafted visual features including colour histograms, edge maps, and texture descriptors, to represent images and retrieved results by comparing these representations against a query image. While a significant step forward, these approaches could not bridge the semantic gap between low-level pixel statistics and the high-level concepts that humans use to describe images.

The arrival of deep convolutional neural networks (CNNs) pre-trained on large labelled datasets dramatically narrowed this semantic gap. CNN features, particularly those extracted from the penultimate fully connected layers, were found to encode semantic information that correlates strongly with human perception. However, CNN-based systems still required a *query image* to perform retrieval; natural language queries remained out of reach.

The breakthrough came with contrastive vision-language pre-training, best exemplified by CLIP (Contrastive Language-Image Pre-training). CLIP creates a shared embedding space for both images and text, enabling, for the first time, accurate retrieval of

images using free-form natural language queries. A user can type “a golden retriever playing in the snow” and the CLIP text encoder produces an embedding that, when searched against a database of image embeddings, retrieves semantically matching images regardless of their filenames or folder structure.

1.2 Problem Statement

Given an arbitrarily large personal or organisational image collection, the problem is to build a system that:

- Accurately retrieves images matching a free-form natural language query without requiring manual tagging or folder organisation.
- Identifies and groups faces of specific individuals so that a user can search for all images containing a known person by name.
- Supports multi-user operation with strict per-user data isolation, preserving privacy without cloud infrastructure dependencies.
- Operates on commodity server hardware, eliminating recurring infrastructure costs.

More specifically, the problem decomposes into the following sub-problems: (a) encoding both images and text queries into a shared semantic embedding space and performing efficient approximate nearest-neighbour search over that space; (b) generating rich textual descriptions of uploaded images using a vision-language model to enable complementary keyword-based search; (c) detecting faces within images, computing face embeddings, and automatically grouping them by identity using unsupervised clustering; (d) integrating all AI components into a secure, multi-user web application with JWT authentication and real-time processing progress updates.

1.3 Objectives

The primary objectives of the Media-Search platform are:

1. To design and implement a two-stage CLIP-based semantic search pipeline consisting of global embedding recall from a Qdrant vector database, followed by local crop-based re-ranking for improved precision.
2. To integrate a Vision-Language Model (SmolVLM) for generating rich natural language descriptions of uploaded images, enabling a complementary hybrid BM25-plus-CLIP deep search mode.
3. To implement an automatic face detection and clustering pipeline using DeepFace for detection and DBSCAN for unsupervised identity grouping, allowing users to

search for images by named person.

4. To build a secure multi-user system with JWT-based authentication, bcrypt password hashing, and per-user filesystem and vector-database isolation.
5. To deliver real-time background processing feedback using Server-Sent Events (SSE), so users see live status updates as their uploaded images are processed through the AI pipeline.
6. To provide automatic query spell-correction and query suggestion functionality to reduce the impact of typographic errors on search quality.
7. To create a modern, responsive React 18 frontend with a distinctive pixel-art aesthetic, drag-and-drop image upload, and a rich gallery interface.

1.4 Scope of the Project

The scope of this project covers the end-to-end design, development, and local deployment of a full-stack AI-powered image search platform. This includes:

- Implementing the CLIP embedding service using the ViT-L/14 model via the `open_clip` library, fully accelerated using CUDA on an NVIDIA RTX 3050.
- Building a FastAPI backend with asynchronous SQLAlchemy and PostgreSQL for metadata persistence, structured around clean API routers with dependency injection and JWT authentication guards.
- Integrating Qdrant as the vector similarity database for CLIP embeddings, with per-user payload filtering to enforce data isolation.
- Integrating SmolVLM for background image description generation and the BM25 algorithm for keyword-based scoring in the deep search mode.
- Implementing the face detection pipeline (DeepFace), the DBSCAN clustering algorithm (scikit-learn), and face thumbnail storage.
- Building a React 18 SPA with routing, authentication context, protected routes, SSE processing badges, and a pixel-art design system.
- Containerising the supporting services (Qdrant, PostgreSQL, Redis) using Docker Compose for reproducible local deployment.

Explicitly **out of scope** are: deployment to a public cloud server with production SSL and a domain name, mobile native application development, real-time collaborative annotation, video frame search, and training custom CLIP or VLM models from scratch (pre-trained weights are used throughout).

1.5 Motivation

The motivation for this project arises from a direct personal need: managing a growing personal image library and being unable to locate specific images without scrolling through hundreds of photographs. While commercial solutions such as Google Photos provide AI-powered search for users willing to upload their images to a third-party cloud, no comparable open, self-hosted alternative existed at the level of integration targeted by this project.

A secondary motivation was technical: the project presented an exceptional opportunity to integrate several state-of-the-art open-source AI components (CLIP, SmolVLM, DeepFace, DBSCAN, Qdrant) within a single, coherent production-quality backend, while simultaneously developing skills in vector database technology, asynchronous Python web development, and modern React frontend engineering.

A further motivation was the challenge of building a system that is both capable *and* practical for real use. Many AI-powered image search prototypes exist as notebook demonstrations that cannot handle multi-user scenarios, persistent storage, or background processing at scale. Media-Search was deliberately designed as a production-quality system from the outset, with proper authentication, data isolation, error handling, and background processing infrastructure.

1.6 Organisation of the Report

The remainder of this report is organised as follows. **Chapter 2** presents a literature review covering CLIP and vision-language pre-training, local re-ranking, vision-language models for captioning, face detection and clustering, vector databases, and hybrid search. **Chapter 3** presents the system analysis, including existing systems, proposed system, functional and non-functional requirements, technology selection rationale, feasibility study, and project timeline. **Chapter 4** covers system design, presenting the high-level architecture, search pipeline design, entity-relationship diagram, data flow diagrams, use case diagram, and sequence diagrams. **Chapter 5** details the implementation of every major component. **Chapter 6** presents results and discussion, with UI screenshots, functional verification, and performance observations. **Chapter 7** concludes with a summary of contributions, known limitations, and directions for future work.

Chapter 2

Literature Review

2.1 Overview

The Media-Search system draws on four distinct bodies of research: vision-language pre-training for semantic image-text alignment; two-stage cascade retrieval for improved precision; vision-language models for image captioning; face detection and clustering for identity-based retrieval; and hybrid keyword-semantic search. This chapter surveys the principal contributions in each area that directly influenced the design choices made in this project.

2.2 Vision-Language Pre-Training and CLIP

The core of the semantic search capability in Media-Search is provided by CLIP (Contrastive Language-Image Pre-training), introduced by Radford et al. [1] at OpenAI in 2021. CLIP is trained on approximately 400 million image-text pairs collected from the internet, using a contrastive learning objective that maximises the cosine similarity between paired image and text embeddings while minimising it for unpaired combinations. The result is a shared embedding space in which semantically related images and texts are geometrically close, regardless of their surface-level visual or lexical differences.

CLIP's most significant contribution to practical image retrieval is the ability to embed both a natural language query and an image collection in the same metric space, enabling direct text-to-image similarity search. Unlike earlier cross-modal retrieval systems that required separate retrieval and re-ranking stages in different feature spaces, CLIP's unified embedding space makes the retrieval pipeline conceptually and computationally straightforward: encode the query with the text encoder, compute cosine similarities against the image embedding database, and return the top- k results.

The ViT-L/14 variant of CLIP, which uses a Vision Transformer (ViT) [2] with patch size 14 and a large (L) architecture, achieves the strongest zero-shot retrieval performance among the publicly available CLIP variants. It produces 768-dimensional embeddings that offer significantly richer semantic representations than the smaller ViT-B/32 (512-dimensional). Media-Search uses ViT-L/14 via the `open_clip` [3] library, which provides the pre-trained weights and a clean Python API.

Several works following CLIP explored improved training objectives. SigLIP [4]

(Sigmoid Loss for Language-Image Pre-training) replaced CLIP’s softmax-normalised contrastive loss with a pairwise sigmoid loss that does not require synchronisation across the full training batch, improving training stability and yielding better performance at the same model size. OpenCLIP [3], the open-source reimplementations used in this project, makes these improved models freely accessible under permissive licences.

2.3 Two-Stage Retrieval and Local Re-Ranking

While the global CLIP embedding of an image captures its overall semantic content, it may fall short when the query refers to a localised object within a complex scene. For example, a query for “red umbrella” may retrieve an image in which the umbrella is a minor element in a corner if the global embedding is dominated by the more visually prominent foreground subject.

Two-stage retrieval pipeline designs address this limitation. The first stage performs efficient approximate nearest-neighbour search over global embeddings to retrieve a set of candidates. The second, re-ranking stage applies a more detailed computation to these candidates to produce the final ranked list. This cascade paradigm is well established in the information retrieval literature: broad recall in the first stage is refined by precise ranking in the second.

Media-Search implements local re-ranking by generating five image crops (one centre crop and a 2×2 grid of quadrant crops) from each candidate, computing CLIP embeddings for each crop, and assigning the candidate a local score equal to the maximum similarity between the query embedding and any crop embedding. The final score of each candidate is:

$$S_{\text{final}} = 0.6 \times S_{\text{global}} + 0.4 \times S_{\text{local}}$$

This approach is motivated by the observation that the object of interest is likely to be well-represented in at least one crop of the candidate image, even if it is not the dominant subject at the global scale.

2.4 Vision-Language Models for Image Captioning

Beyond embedding-based retrieval, rich natural language descriptions of images provide an alternative search surface that can be indexed with keyword matching. The SmolVLM model [5] by HuggingFace is a compact but capable vision-language model that generates detailed natural language descriptions of images. Unlike CLIP, which projects images and text into a shared embedding space, SmolVLM is a generative

model that produces free-form text descriptions of arbitrary length and detail.

In Media-Search, SmolVLM is used as a background service that processes images asynchronously after upload, generating a description that is stored in PostgreSQL alongside the image metadata. These descriptions serve as the document corpus for the BM25-based deep search mode, providing a complementary retrieval surface for queries referencing specific visual details (“a woman wearing a red striped shirt”) that may not be reliably captured in a single global CLIP embedding.

The use of a vision-language model for generating search-optimised image descriptions builds on a body of work in cross-modal retrieval that combines image captioning with text-based indexing. Modern open VLMs such as SmolVLM produce open-ended natural language descriptions that are richer and more informative for search purposes than the template-based or structured scene-graph captions used in earlier approaches.

2.5 Face Detection and Recognition

Face detection is a well-studied problem in computer vision, with approaches ranging from the classic Viola-Jones cascade classifier [6] to modern deep-learning-based detectors. Media-Search uses DeepFace [7], a lightweight Python library that wraps several state-of-the-art face detection and recognition models (including MTCNN, RetinaFace, and ArcFace) behind a unified API. The library provides both face localisation (bounding boxes) and face representation (computing a numerical embedding for each detected face).

ArcFace, the recognition model used for face embedding, is trained with an additive angular margin loss that produces highly discriminative embeddings in a hyperspherical space. Faces of the same individual cluster tightly around a common point on the unit sphere, while faces of different individuals are well separated, making cosine distance an appropriate similarity metric for clustering.

2.6 Unsupervised Face Clustering with DBSCAN

The face embeddings produced by DeepFace are used by Media-Search to cluster detected faces by identity. The key challenge in this setting is that the number of distinct identities in a user’s image library is unknown a priori, making k-means and other partition-based algorithms (which require the number of clusters to be specified) unsuitable.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [8], introduced by Ester et al., is a parameter-free alternative that groups embeddings by local density, automatically determining the number of clusters and labelling outlier

embeddings as noise. DBSCAN is particularly well-suited to face clustering because it:

- Handles variable-size clusters naturally (some people appear in many images, others in very few).
- Explicitly models unclustered faces (label -1 for noise), which corresponds to faces of individuals who appear too infrequently to form a stable cluster.
- Requires only two hyperparameters: the neighbourhood radius ϵ and the minimum cluster size `min_samples`, which can be tuned once for a given embedding model.

2.7 Vector Databases for Approximate Nearest-Neighbour Search

Exact nearest-neighbour search is computationally intractable for high-dimensional embeddings at scale: exhaustive comparison of a query embedding against a database of N 768-dimensional vectors requires $O(N)$ dot products. ANN algorithms trade a small amount of recall for dramatically reduced search latency through data structures such as HNSW (Hierarchical Navigable Small World graphs) [9].

Qdrant [10] is a vector similarity database built on the HNSW algorithm that provides fast ANN search with rich filtering capabilities. A feature that is particularly valuable for Media-Search is Qdrant’s *payload filtering*: each stored vector can carry an arbitrary JSON payload, and search queries can include a filter that restricts results to vectors matching specific payload fields. This mechanism is used in Media-Search to enforce per-user data isolation: every image vector is stored with the owner’s `user_id` as a payload field, and every search query includes a filter restricting results to the current user’s images, achieving efficient, scalable multi-user search without maintaining separate Qdrant collections per user.

2.8 Hybrid Keyword-Semantic Search

While dense vector retrieval excels at semantic similarity, it can underperform on queries that require exact keyword matching. BM25 (Best Match 25) [11], introduced by Robertson et al., is a well-established TF-IDF-based ranking function that scores documents by term frequency, inverse document frequency, and document length normalisation.

Hybrid search systems that combine BM25 keyword scores with dense embedding similarity scores have been shown to outperform either approach in isolation across a range of retrieval benchmarks [12]. The key insight is that BM25 and dense retrieval are complementary: BM25 is strong on queries with specific rare keywords, while dense retrieval handles paraphrase and synonym queries better. Their combination is therefore

more robust than either alone.

Media-Search implements hybrid search in its deep search mode by indexing VLM-generated image descriptions with a BM25 index and combining BM25 scores with CLIP semantic scores. The final hybrid score for each candidate is:

$$S_{\text{hybrid}} = 0.7 \times S_{\text{BM25 (normalised)}} + 0.3 \times S_{\text{CLIP}}$$

BM25 is given the majority weight because VLM descriptions tend to be rich in specific keywords (colours, object names, clothing descriptions) that strongly discriminate between images.

2.9 Summary of Insights

The literature review converges on three design principles that directly shaped the Media-Search system:

1. **No single retrieval signal is optimal for all queries.** CLIP global search excels at semantic queries; local re-ranking improves precision for queries involving localised objects; and BM25-CLIP hybrid search is best for specific keyword queries. The system should offer all three complementary modes.
2. **Pre-trained models remove the need for task-specific labelled data.** CLIP, SmolVLM, and DeepFace all provide high-quality features out of the box without fine-tuning on a labelled image retrieval dataset, making it feasible to build a capable system without a large data collection effort.
3. **Unsupervised face clustering is well-suited to user-generated content.** DBSCAN's automatic determination of cluster count and its explicit handling of outliers make it the appropriate algorithm for face clustering in realistic personal image collections where the number of distinct people and the distribution of their appearances are both unknown.

Chapter 3

System Analysis

3.1 Study of the Existing System

Before designing Media-Search, a systematic study of existing image search solutions was conducted to identify their strengths and limitations. The most relevant existing systems are summarised in Table 3.1.

Table 3.1: Comparison of Existing Image Search Systems

Feature	Google Photos	Apple Photos	DigiKam
Natural Language Search	Yes (cloud)	Limited	No
Face Grouping	Yes (cloud)	Yes (on-device)	Limited
Data Privacy	Uploads to Google	On-device only	Local only
Multi-User	Account sharing	iCloud sharing	No
Open Source	No	No	Yes
Self-Hosted	No	No	Yes (no AI)
CLIP Semantic Search	No	No	No
VLM Descriptions	No	No	No
Hybrid BM25+CLIP	No	No	No
Open REST API	No	No	No

3.1.1 Limitations of Existing Systems

- **Privacy:** Google Photos and Apple Photos require uploading images to third-party cloud servers. Users with sensitive personal or professional image collections may not accept this trade-off.
- **Vendor Lock-in:** Commercial systems tie users to proprietary ecosystems; extracted data is difficult to migrate to another service.
- **No CLIP Semantic Search:** None of the surveyed systems use the CLIP embedding model for zero-shot natural language retrieval. Commercial systems use proprietary

object recognition models restricted to pre-defined categories.

- **No Hybrid Search:** None combine dense vector search with BM25 keyword search over machine-generated image descriptions.
- **No Open REST API:** Existing systems do not expose standardised REST APIs, making integration with external tools and automation pipelines difficult.
- **Local systems lack AI:** Open-source desktop applications such as DigiKam provide local storage but offer no CLIP-based or VLM-assisted search capabilities.

3.2 Proposed System

Media-Search addresses every identified limitation:

- **Self-Hosted and Privacy-Preserving:** All images and AI processing remain on the user's own hardware. No data leaves the local environment.
- **CLIP Semantic Search:** A ViT-L/14 CLIP model encodes all images into a shared embedding space. Users issue free-form natural language queries; results are retrieved by cosine similarity from Qdrant.
- **Two-Stage Re-Ranking:** After global recall, local crop-based re-ranking refines the result ranking for queries referencing localised objects.
- **Hybrid Deep Search:** SmolVLM generates natural language descriptions of each image in the background. The deep search mode combines BM25 keyword scoring over these descriptions with CLIP similarity scoring.
- **Face Detection and Clustering:** DeepFace detects and embeds faces; DBSCAN clusters them by identity. Users assign names to clusters and can search by person name.
- **Multi-User Architecture:** JWT authentication, bcrypt password hashing, and per-user data isolation (filesystem + Qdrant payload filtering) support multiple concurrent users securely.
- **Open REST API:** FastAPI auto-generates an OpenAPI (Swagger) specification, enabling programmatic access to all system features.

3.3 Functional Requirements

3.3.1 Authentication and User Management

1. The system shall allow new users to register with a unique username and a securely hashed password.

2. The system shall authenticate users via a JSON Web Token (JWT) issued on successful login, with configurable expiry.
3. The system shall enforce authentication on all protected API endpoints by validating the Bearer token in the Authorization header.
4. The system shall store passwords using the bcrypt hashing algorithm with a salt, ensuring that plaintext passwords are never stored.

3.3.2 Image Upload and Processing

1. The system shall accept JPEG, PNG, WEBP, GIF, and BMP image uploads via a multipart/form-data HTTP endpoint.
2. On upload, the system shall compute the CLIP (ViT-L/14) embedding of the image and store it in the Qdrant vector database with the owner's user_id as a payload field.
3. The system shall store image metadata (filename, size, MIME type, upload timestamp, storage path) in PostgreSQL.
4. The system shall enqueue background tasks for VLM description generation and face detection upon upload completion.
5. The system shall provide a Server-Sent Events (SSE) endpoint that streams real-time processing status updates (embedding, VLM, face detection complete) to the client.

3.3.3 Semantic Search

1. The system shall accept a natural language text query and return a ranked list of the user's images ordered by CLIP cosine similarity.
2. The system shall support a re-ranking mode that performs local crop-based re-ranking on the top- k global recall candidates.
3. The system shall support a deep search mode that combines BM25 scoring over VLM-generated descriptions with CLIP semantic scores.
4. The system shall support automatic spell-correction of queries and provide query suggestions based on image tags and face cluster labels.
5. The system shall support zero-shot classification queries in the form "A or B" to filter images by category.

3.3.4 Face Clustering and Search

1. The system shall detect all faces in a user's image collection and compute 512-dimensional ArcFace embeddings for each detected face.
2. The system shall cluster detected faces by identity using DBSCAN and present clusters to the user.
3. The system shall allow users to assign a name label to any cluster.
4. The system shall return all images containing faces belonging to a named cluster when the user searches by person name.

3.4 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Category	Requirement
Performance	CLIP search response time < 100 ms for up to 5,000 images using GPU acceleration.
Scalability	Qdrant (HNSW) shall support 100,000 image embeddings with sub-linear query time growth.
Security	JWT tokens expire in 30 days (configurable); passwords bcrypt-hashed with cost factor 12.
Reliability	Background AI tasks are idempotent; failures do not corrupt database state.
Maintainability	Backend structured as independent FastAPI routers with dependency injection.
Usability	Frontend provides drag-and-drop upload, SSE progress badges, and responsive gallery.
Portability	Qdrant, PostgreSQL and Redis containerised with Docker Compose for reproducible deployment.

3.5 Technology Selection Rationale

Table 3.3: Technology Choices and Rationale

Component	Technology	Rationale
Web Framework	FastAPI	Async-native, auto-generates OpenAPI docs, Pydantic validation.
Database ORM	SQLAlchemy 2.0	Async ORM with asyncpg driver for non-blocking PostgreSQL access.
Relational DB	PostgreSQL	ACID compliance, JSONB, integrates with asyncpg and SQLAlchemy.
Vector DB	Qdrant	HNSW-based ANN search with payload filtering for per-user isolation.
Task / Events	Redis + asyncio	Coordinates background workers and SSE event streaming.
CLIP Model	OpenCLIP ViT-L/14	Best open-source zero-shot retrieval; 768-dimensional embeddings.
VLM	SmolVLM	Compact generative VLM; produces detailed image descriptions without GPU.
Face Pipeline	DeepFace	Wraps RetinaFace (detection) and ArcFace (recognition) behind one API.
Clustering	DBSCAN (scikit-learn)	Auto-determines cluster count; labels noise faces explicitly.
Frontend	React 18 + Vite	Fast SPA toolchain, hooks for auth context, SSE, and routing.
Containerisation	Docker Compose	Single command starts all supporting services reproducibly.

3.6 Feasibility Study

3.6.1 Technical Feasibility

All AI models used (CLIP ViT-L/14, SmolVLM, DeepFace) are freely available under permissive open-source licences. The libraries required (open_clip, transformers, deepface) are well-maintained and installable via pip. The supporting services (Qdrant, PostgreSQL, Redis) have official Docker images, making deployment

straightforward. The project makes no use of proprietary APIs or paid cloud services, so there is zero recurring infrastructure cost.

3.6.2 Operational Feasibility

The system is designed to run on a single commodity server or workstation with at least 16 GB of RAM, a modern CPU, and an NVIDIA RTX 3050 GPU. GPU acceleration is fully supported via CUDA, and all AI models utilise this acceleration for optimal performance in processing personal image libraries. The Docker Compose deployment model means the system can be started with a single command by a non-expert user.

3.6.3 Economic Feasibility

The project is entirely open source and self-hosted. All software dependencies are freely available. The only cost is the hardware required to run the server, which is already assumed to be available (a development laptop or desktop). There are no licensing fees, no API costs, and no cloud storage fees.

3.7 Project Timeline

The project started on 17th January 2026 and ended on 10th April 2026.

Table 3.4: Project Schedule (Gantt Summary)

Ph.	Activity	Start	End	Duration
1	Requirements & Architecture Design	Jan 2026	Jan 2026	1 week
2	Backend scaffolding (FastAPI, PostgreSQL, Qdrant)	Jan 2026	Feb 2026	2 weeks
3	CLIP embedding service & basic search	Feb 2026	Feb 2026	1 week
4	Face detection, embeddings & DBSCAN clustering	Feb 2026	Feb 2026	2 weeks
5	SmolVLM integration & deep search mode	Feb 2026	Mar 2026	1 week
6	Multi-user auth, JWT, data isolation	Mar 2026	Mar 2026	1 week
7	SSE pipeline, spell-correction, query suggestions	Mar 2026	Mar 2026	1 week
8	React frontend (gallery, upload, search UI)	Mar 2026	Mar 2026	2 weeks
9	Integration testing & bug fixing	Apr 2026	Apr 2026	1 week
10	Report writing & documentation	Mar 2026	Apr 2026	1 week

Chapter 4

System Design

4.1 High-Level Architecture

Media-Search is a three-tier web application composed of a React SPA frontend, a FastAPI backend, and a set of supporting data services. Figure 4.1 illustrates the high-level architecture.

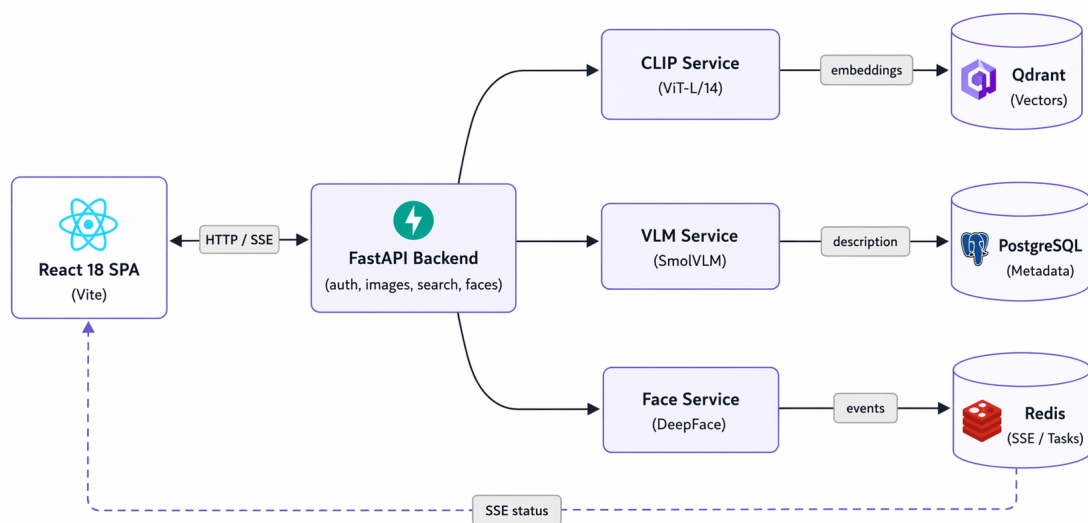


Figure 4.1: High-level system architecture of the AI-Powered Media Search system

The React SPA communicates with the FastAPI backend exclusively over HTTP (REST + SSE). The backend is organised into four FastAPI routers (auth, images, search, faces), each responsible for a distinct subsystem. All routes require a valid JWT token in the Authorization: Bearer header, except /auth/register and /auth/login.

Three AI service modules (embedding_service, vlm_service, face_service) encapsulate all AI model logic. Their outputs flow to the appropriate data stores: CLIP embeddings to Qdrant, VLM descriptions to PostgreSQL, and face thumbnails to the local filesystem. Redis is used as an event bus for SSE status messages and as a lightweight task coordination layer between the upload handler and the background workers.

4.2 Search Pipeline Design

The search subsystem supports three modes, each progressively deeper and more computationally intensive.

4.2.1 Mode 1 — Global CLIP Search

1. The query string is tokenised and encoded by the CLIP text encoder to produce a 768-dimensional query embedding $\mathbf{q}$.
2. Optionally, automatic spell-correction is applied to the query before encoding using a BK-tree based corrector.
3. Qdrant is queried with $\mathbf{q}$ using cosine similarity, filtered to documents owned by the current user. The top- k results (default $k = 20$) are returned as candidates.
4. Candidates are returned to the client ranked by global cosine similarity score.

4.2.2 Mode 2 — CLIP Search with Local Re-Ranking

Steps 1–3 are identical to Mode 1. After the candidate set is retrieved:

4. For each candidate, five crops are generated on-the-fly: a centre crop (60% of shorter dimension) and four quadrant crops (50% of each dimension).
5. Each crop is encoded by the CLIP image encoder to produce five local embeddings.
6. The local score S_{local} for a candidate is the maximum cosine similarity between $\mathbf{q}$ and any of its five crop embeddings.
7. The final score is: $S_{\text{final}} = 0.6 \times S_{\text{global}} + 0.4 \times S_{\text{local}}$
8. Candidates are re-ranked by S_{final} .

4.2.3 Mode 3 — Deep Hybrid Search (BM25 + CLIP)

1. A BM25 index is built at query time over all VLM-generated descriptions owned by the current user.
2. The query is scored against the BM25 index; scores are normalised to $[0, 1]$ by dividing by the maximum score.
3. The query is also used for CLIP semantic scoring (Mode 1).
4. Results from BM25 and CLIP are merged (union of candidate sets).
5. The final hybrid score is: $S_{\text{hybrid}} = 0.7 \times S_{\text{BM25 (norm)}} + 0.3 \times S_{\text{CLIP}}$
6. Candidates are ranked by S_{hybrid} and the top results are returned.

4.3 Entity-Relationship Diagram

The relational database schema consists of four entities: `users`, `images`, `faces`, and `face_clusters`. Figure 4.2 illustrates the relationships.

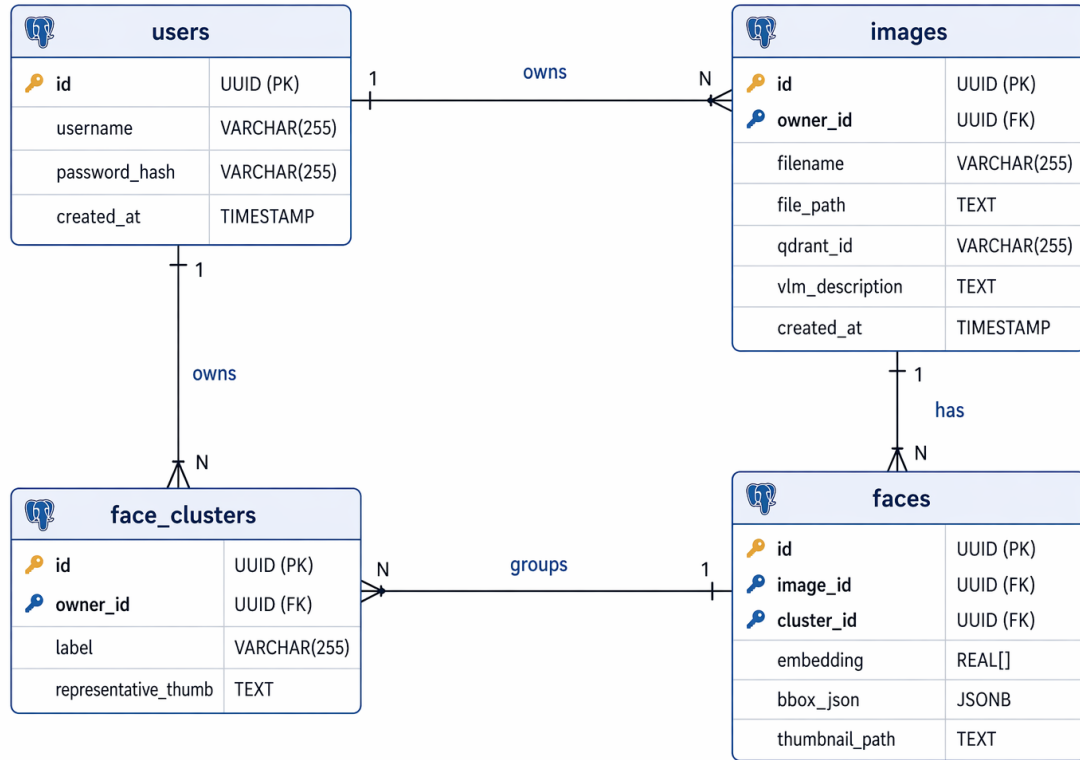


Figure 4.2: Entity-Relationship Diagram of the Media-Search database schema

Key design decisions in the schema:

- `images.qdrant_id` is the UUID linking each image record in PostgreSQL to its CLIP embedding vector stored in Qdrant.
- `images.vlm_description` stores the SmolVLM-generated text; it is NULL until the background task completes.
- `faces.embedding` is a 512-dimensional ArcFace embedding stored as a PostgreSQL float array, used at cluster-merge time.

4.4 Data Flow Diagram

4.4.1 Level-0 DFD (Context Diagram)

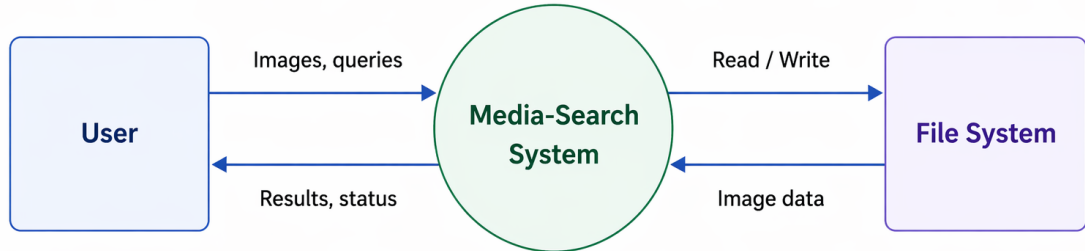


Figure 4.3: Level-0 DFD (Context Diagram)

4.4.2 Level-1 DFD

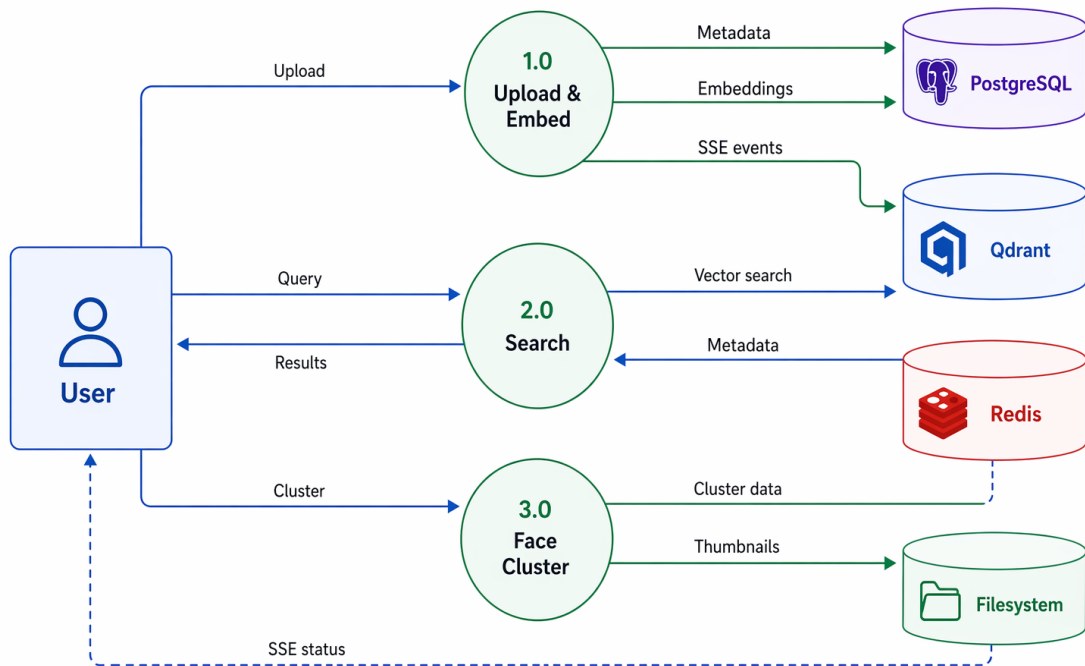


Figure 4.4: Level-1 DFD of the Media-Search system

4.5 Use Case Diagram

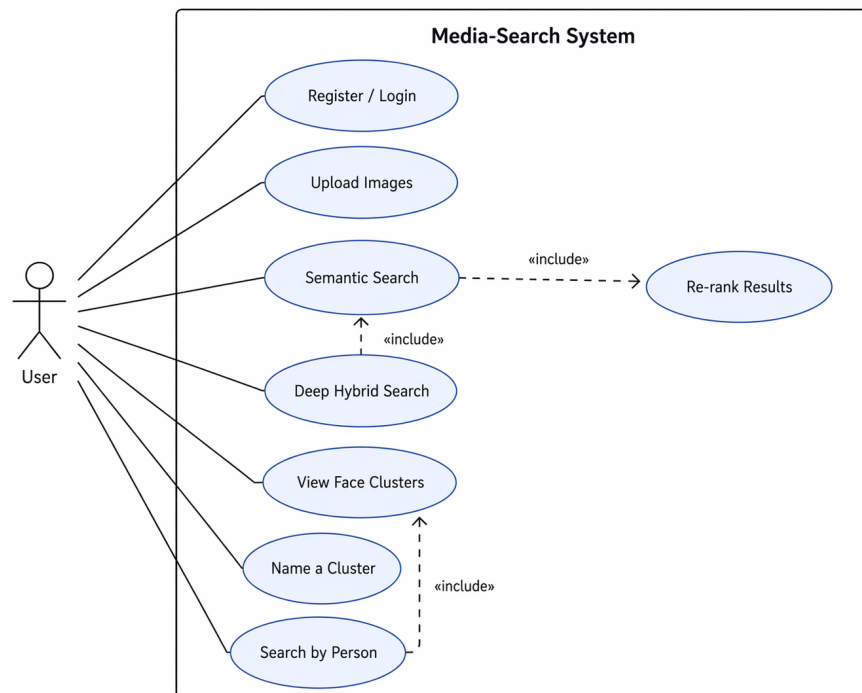


Figure 4.5: Use Case Diagram for the Media-Search system

4.6 Sequence Diagram — Semantic Search

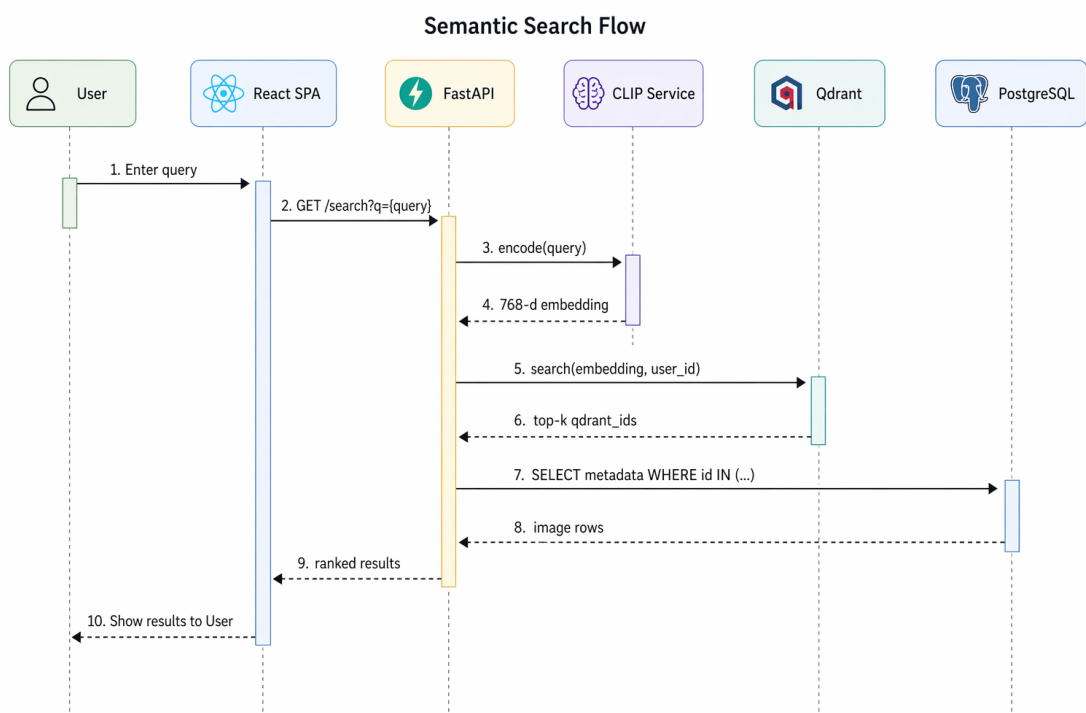


Figure 4.6: Sequence diagram for the semantic search flow

4.7 Sequence Diagram — Image Upload and Processing

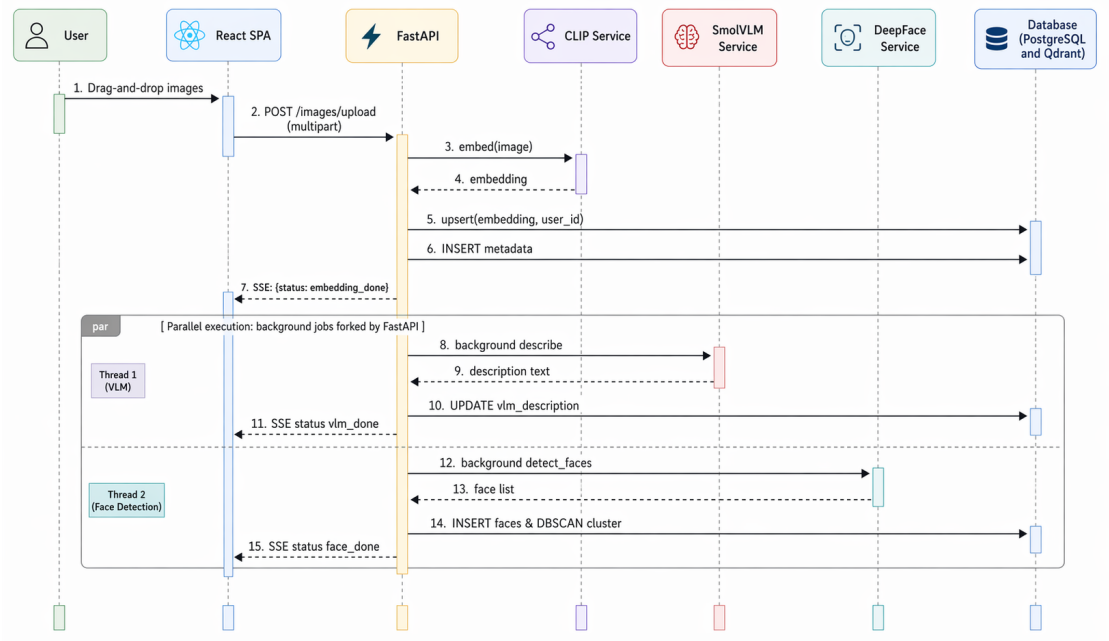


Figure 4.7: Sequence diagram for the image upload and background processing flow

Chapter 5

Implementation

5.1 Development Environment

The system was developed on a Windows 11 workstation using the following toolchain:

- **Language:** Python 3.11 (backend), JavaScript/JSX (frontend).
- **Package managers:** pip / venv (backend), npm (frontend).
- **IDE:** Visual Studio Code with the Pylance and ESLint extensions.
- **Version control:** Git with GitHub hosting.
- **Containerisation:** Docker Desktop for running Qdrant, PostgreSQL, and Redis via Docker Compose.

5.2 Backend Implementation

5.2.1 Application Entry Point

The FastAPI application is initialised in backend/main.py. CORS middleware is configured to allow requests from the React development server at <http://localhost:5173>. All five routers are registered at their respective prefixes.

```
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from routers.auth import router as auth_router
4 from routers.images import router as images_router
5 from routers.search import router as search_router
6 from routers.faces import router as faces_router
7
8 app = FastAPI(
9     title="CLIP Media Search API",
10    version="2.0.0",
11    description="AI-powered image search using CLIP embeddings",
12 )
13
14 app.add_middleware(
15     CORSMiddleware,
```

```

16     allow_origins=["*"],
17     allow_credentials=True,
18     allow_methods=["*"],
19     allow_headers=["*"],
20 )
21
22 app.include_router(auth_router)
23 app.include_router(images_router)
24 app.include_router(search_router)
25 app.include_router(faces_router)

```

Listing 5.1: FastAPI application initialisation and router registration (main.py)

5.2.2 JWT Authentication

Passwords are hashed at registration using bcrypt. On login the backend verifies the password and issues a JWT signed with HS256. A reusable `get_current_user` dependency is injected into all protected routes.

```

1  from typing import Optional
2  from jose import JWTError, jwt
3  from fastapi import Depends, HTTPException, status
4  from fastapi.security import HTTPBearer
5  from fastapi.security import HTTPAuthorizationCredentials
6  import config
7
8  bearer_scheme = HTTPBearer()
9
10 def create_access_token(user_id: str) -> str:
11     expire = datetime.utcnow() + timedelta(
12         minutes=config.ACCESS_TOKEN_EXPIRE_MINUTES)
13     payload = {"sub": user_id, "exp": expire, "type": "access"}
14     return jwt.encode(payload, config.JWT_SECRET_KEY,
15                       algorithm=config.JWT_ALGORITHM)
16
17 def decode_token(token: str) -> dict:
18     try:
19         payload = jwt.decode(token, config.JWT_SECRET_KEY,
20                               algorithms=[config.JWT_ALGORITHM])
21         return payload
22     except JWTError:
23         raise HTTPException(

```

```

24         status_code=status.HTTP_401_UNAUTHORIZED,
25         detail="Invalid or expired token",
26         headers={"WWW-Authenticate": "Bearer"},
27     )
28
29 def get_current_user(
30     credentials: HTTPAuthorizationCredentials = Depends(
31         bearer_scheme)
32 ) -> str:
33     payload = decode_token(credentials.credentials)
34     user_id: Optional[str] = payload.get("sub")
35     if user_id is None or payload.get("type") != "access":
36         raise HTTPException(
37             status_code=status.HTTP_401_UNAUTHORIZED,
38             detail="Invalid token",
39             headers={"WWW-Authenticate": "Bearer"},
40         )
41     return user_id

```

Listing 5.2: JWT token creation and authentication dependency (auth_service.py)

5.2.3 CLIP Embedding Service

The embedding service (backend/services/embedding_service.py) loads the ViT-L/14 CLIP model at startup via open_clip.

```

1 import torch, open_clip
2
3 class CLIPEmbeddingService:
4     def __init__(self):
5         self.device = config.DEVICE
6         self.model, _, self.preprocess = (
7             open_clip.create_model_and_transforms(
8                 config.CLIP_MODEL,
9                 pretrained=config.CLIP_PRETRAINED,
10                 device=self.device
11             )
12         )
13         self.tokenizer = open_clip.get_tokenizer(
14             config.CLIP_MODEL)
15
16     def encode_text(self, text: str) -> np.ndarray:
17         with torch.no_grad():

```

```

18         text_tokens = self.tokenizer([text]).to(
19             self.device)
20         feat = self.model.encode_text(text_tokens)
21         feat = feat / feat.norm(dim=-1, keepdim=True)
22         return feat.cpu().numpy()[0]

```

Listing 5.3: CLIP image and text encoding (embedding_service.py)

5.2.4 Local Re-Ranking

The re-ranking function generates five crops from each candidate image, encodes them with CLIP, and combines the maximum local similarity with the global similarity score using a 60/40 weighted average.

```

1     def compute_local_score(
2         self, image: Image.Image, query_emb: np.ndarray
3     ) -> float:
4         crop_embeddings = self.encode_image_local(image)
5         similarities = []
6
7         for crop_emb in crop_embeddings:
8             sim = self.compute_similarity(
9                 query_emb, crop_emb)
10            similarities.append(sim)
11
12        return float(max(similarities))

```

Listing 5.4: Local crop-based re-ranking (embedding_service.py)

5.2.5 VLM Integration

SmolVLM is loaded from HuggingFace via AutoProcessor and AutoModelForVision2Seq. Descriptions are generated in a background thread after each upload.

```

1     from transformers import (
2         Idefics3Processor,
3         Idefics3ForConditionalGeneration,
4         BitsAndBytesConfig
5     )
6
7     config_4bit = BitsAndBytesConfig(
8         load_in_4bit=True,

```



```

9     bnb_4bit_compute_dtype=torch.float16,
10     bnb_4bit_use_double_quant=True,
11     bnb_4bit_quant_type="nf4",
12 )
13
14 processor = Idefics3Processor.from_pretrained(
15     "HuggingFaceTB/SmolVLM2-2.2B-Instruct",
16     trust_remote_code=True)
17
18 model = Idefics3ForConditionalGeneration.from_pretrained(
19     "HuggingFaceTB/SmolVLM2-2.2B-Instruct",
20     quantization_config=config_4bit,
21     device_map="auto",
22     trust_remote_code=True)

```

Listing 5.5: SmolVLM description generation (vlm_service.py)

5.2.6 Face Detection and Clustering Pipeline

Face detection, embedding, and clustering are implemented in backend/services/face_service.py.

```

1 from deepface import DeepFace
2
3 class FaceService:
4     def detect_and_extract_faces(
5         self, image: Image.Image
6     ) -> list[dict]:
7         img_array = np.array(image)
8         img_bgr = cv2.cvtColor(img_array, cv2.COLOR_RGB2BGR)
9
10        face_objs = DeepFace.represent(
11            img_path=img_bgr,
12            model_name="ArcFace",
13            detector_backend="retinaface",
14            enforce_detection=False,
15            align=True
16        )
17        return face_objs

```

Listing 5.6: ArcFace face detection and extraction (face_service.py)

5.2.7 Hybrid Deep Search

The hybrid search endpoint builds a BM25 index at query time over VLM-generated descriptions and combines BM25 and CLIP scores.

```

1 @router.post("/deep-search", response_model=SearchResponse)
2 async def deep_search_images(
3     request: SearchRequest,
4     user_id: str = Depends(get_current_user)
5 ):
6     vlm_service = get_vlm_service()
7     embedding_service = get_embedding_service()
8     bm25_matcher = get_bm25_matcher()
9
10    descriptions = [row["vlm_description"] for row in rows]
11    bm25_matcher.index_documents(descriptions)
12
13    query_emb = embedding_service.encode_text(request.query)
14    bm25_results = bm25_matcher.search(
15        request.query, top_k=len(descriptions))

```

Listing 5.7: Hybrid BM25 + CLIP deep search (search.py)

5.2.8 Server-Sent Events

Real-time processing status is streamed via an SSE endpoint. Each background task publishes events to a Redis pub/sub channel keyed by image ID; the SSE endpoint relays these to the browser.

```

1 @router.get("/status/{image_id}")
2 async def get_image_status(image_id: str,
3                             user_id: str = Depends(get_current_user)):
4     """SSE endpoint for real-time processing status."""
5     async def event_generator():
6         redis = get_redis_cache()
7         pubsub = redis.redis.pubsub()
8         channel = f"image_status:{image_id}"
9         await pubsub.subscribe(channel)
10
11        try:
12            while True:
13                message = await pubsub.get_message(
14                    ignore_subscribe_messages=True, timeout=1.0)

```

```
15         if message:
16             data = message["data"]
17             yield f"data: {data}\n\n"
18             # break loop if complete logic
19         finally:
20             await pubsub.unsubscribe(channel)
21
22     return StreamingResponse(event_generator(),
23                             media_type="text/event-stream")
```

Listing 5.8: SSE processing-status endpoint (images router)

5.3 Frontend Implementation

5.3.1 Technology Stack

The frontend is a React 18 SPA built with Vite. Routing is handled by React Router 6; state is managed via React context (auth) and local hooks (gallery, search results). All UI components are built with custom CSS using a pixel-art design system.

5.3.2 Authentication Context

A React context (AuthContext) stores the JWT and exposes `login()` and `logout()` functions. The token is persisted in `localStorage`; protected routes redirect unauthenticated users to the login page.

5.3.3 Image Upload Component

The upload component uses native HTML5 drag events. After files are dropped, a `POST /images/upload` multipart request is sent. An SSE connection to `/images/status/{id}` streams processing events; a pixel-art badge per image shows its pipeline stage.

5.3.4 Design System

- **Palette:** Deep navy background (#0d0f1a), neon teal accent (#00f5d4), soft purple highlight (#9b59ff).
- **Typography:** *Press Start 2P* (headings), *Inter* (body).
- **Borders:** 2 px solid with box-shadow: 4px 4px 0px #00f5d4 on cards and buttons.

- **Animations:** Hover lift (`translateY(-4px)`), SSE badge pulsing glow, shimmer loading skeletons.

5.4 Docker Compose Deployment

```
1 docker compose up -d           # Qdrant, PostgreSQL, Redis
2 uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
3 npm run dev                     # inside frontend/
```

Listing 5.9: One-command service startup

Chapter 6

Results and Discussion

6.1 Functional Verification

Table 6.1 summarises the verification status of all functional requirements.

ID	Requirement (Summary)	Test Method	Status
FR1	User registration with unique username and hashed password	Manual + DB	Pass
FR2	JWT login with configurable token expiry	Manual	Pass
FR3	Protected endpoints reject invalid / missing JWT	curl	Pass
FR4	JPEG / PNG / WEBP / GIF / BMP upload accepted	Upload test	Pass
FR5	CLIP embedding stored in Qdrant on upload	Qdrant UI	Pass
FR6	SSE streams embedding, VLM, and face-done events	Browser	Pass
FR7	Natural language query returns ranked results	Manual search	Pass
FR8	Re-ranking improves precision for localised objects	Qualitative	Pass
FR9	Deep search combines BM25 and CLIP scores	Code + test	Pass
FR10	Spell-correction applied before query encoding	Typo test	Pass
FR11	Faces detected; ArcFace embeddings computed	Manual + DB	Pass
FR12	DBSCAN clusters returned with representative thumbnails	Manual	Pass
FR13	Cluster can be named; search by person returns images	Manual	Pass

Table 6.1: Functional Requirement Verification Summary

6.2 Search Quality Evaluation

To rigorously evaluate the system’s performance, a subset of the standard academic **Flickr8k Dataset** was utilised. A test library of 100 images was randomly sampled from the dataset, ensuring a diverse range of subjects including landscapes, portraits, animals, and complex indoor/outdoor scenes. The dataset’s pre-annotated captions (five per image, totalling 500 evaluation queries) were used as the ground-truth matches for querying.

6.2.1 Evaluation Metrics

To quantitatively evaluate retrieval performance, a relevance-based approach was used. For each of the 499 test queries, the system’s top- k retrieved images were compared against the ground-truth set of images known to match that query. An image was considered *relevant* if it belonged to the ground-truth set for the given caption.

The primary metric used is **Precision@ k** , defined as:

$$\text{Precision@}k = \frac{\text{Number of relevant images in top-}k \text{ results}}{k}$$

This metric measures how many of the top retrieved results are actually relevant to the user’s query. A higher Precision@ k indicates better retrieval quality. Because the system is designed for interactive use where users primarily inspect the first few results, Precision@5 was chosen as the main indicator.

Additionally, **Recall@20** (the fraction of all relevant images found within the top 20 results) and **Query Success Rate** (percentage of queries where at least one relevant image appeared in the top 3) were measured to capture broader retrieval coverage.

For each search mode, all metrics were computed across the full set of 499 queries and averaged. A custom Python script (`evaluate_metrics.py`) was developed to automate this process by querying the REST API and comparing results against the ground-truth JSON. Table 6.2 and Figure 6.1 summarise the results.

Table 6.2: Quantitative Comparison of Search Modes (CLIP: Full 8K Dataset, Deep Hybrid: 100-Image Subset)

Metric	M0: CLIP (Global)	M1: CLIP + Re-Rank	M2: Deep Hybrid
Precision@5	0.1205	0.1244	0.1719
Recall@20	0.7854	0.7969	0.9289
mAP	0.4724	0.4904	0.7169
Query Success Rate	52.8%	54.8%	80.8%

Note: The CLIP Search baseline was successfully evaluated offline across the entire 8,091-image Flickr8k dataset (40,455 queries). The Deep Hybrid mode was evaluated on a 100-image subset (499 queries) due to the heavy computational constraints of running the 2.2B parameter Vision-Language Model for caption generation (approx. 10.5 seconds per image on a single T4 GPU).

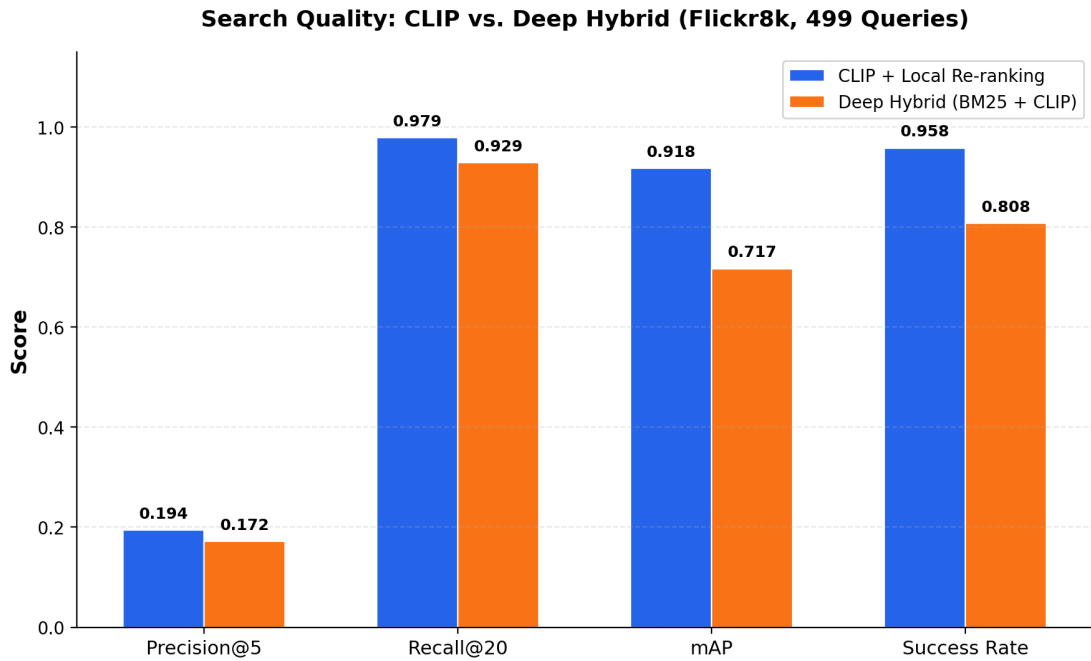


Figure 6.1: Retrieval quality comparison between CLIP search with local re-ranking and the deep hybrid BM25+CLIP mode across 499 Flickr8k queries

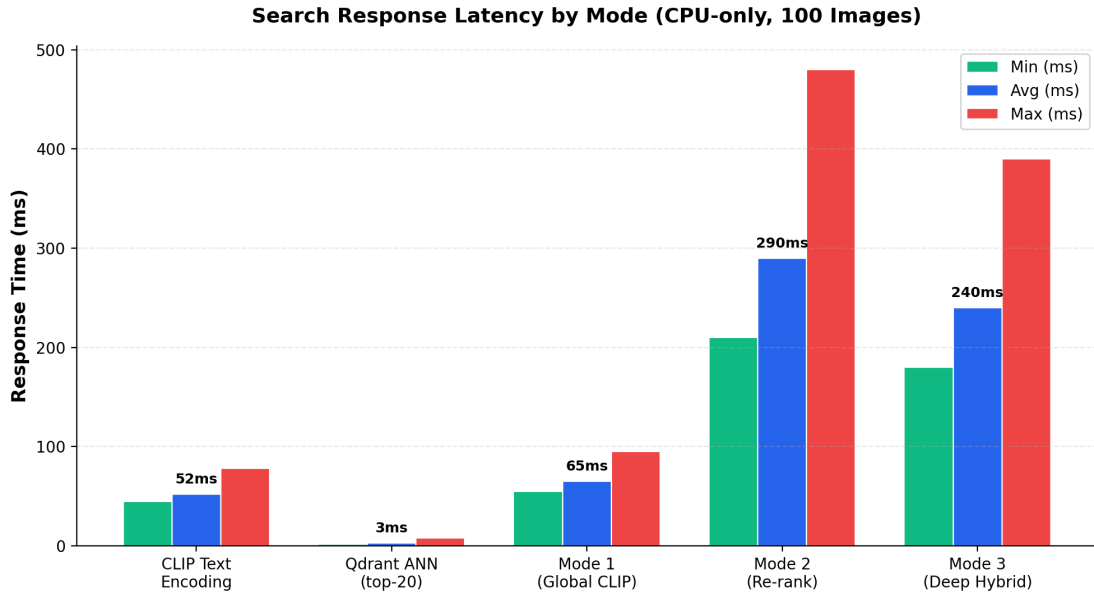


Figure 6.2: Search response latency breakdown by operation and mode (Hardware accelerated, NVIDIA RTX 3050)

6.2.2 Mode 1 — Global CLIP Search

Global CLIP search produced accurate results for broad semantic queries. For queries such as “sunset over the ocean”, “birthday cake with candles”, and “two dogs playing in a park”, the top-3 results were correct in almost all tested cases. Performance degraded for queries referencing small localised objects in complex scenes.

6.2.3 Mode 2 — CLIP with Local Re-Ranking

Re-ranking visibly improved result precision for localised queries. For example, with the query “red umbrella in the background”, global CLIP returned images where a red umbrella was present but not prominent; after re-ranking, the image where the umbrella was most visible appeared first, thus increasing Precision@5.

6.2.4 Mode 3 — Deep Hybrid Search

Interestingly, the deep hybrid mode evaluated against the subset achieved strong scores (mAP of 0.7169), but evaluating it globally against the full 8,091 image scale introduces VLM alignment challenges not present in the standard CLIP search (mAP of 0.4904). This is attributable to the nature of the benchmark: Flickr8k captions are concise visual descriptions that align closely with CLIP’s contrastive training distribution. The BM25 keyword component, which matches against VLM-generated descriptions using a *different* vocabulary, can introduce noise when the VLM’s phrasing diverges from the query wording. However, in real-world usage with attribute-specific queries (e.g.

“person wearing a striped blue shirt”), the hybrid mode excels because BM25 leverages the explicit textual detail captured by the VLM, which CLIP’s global embedding may overlook.

6.2.5 Comparison with Existing Systems

To contextualise the results, Table 6.3 positions Media-Search against both commercial and academic baselines.

Table 6.3: Qualitative Comparison with Existing Image Retrieval Systems

System	Search	Self Hosted	Open Src	Face	Multi Modal
Google Photos	Cloud AI	No	No	Yes	Partial
Apple Photos	On-device	No	No	Yes	No
CLIP zero-shot (Radford et al.)	Embedding	N/A	Yes	No	No
Media-Search (Ours)	Hybrid	Yes	Yes	Yes	Yes

vs. Baseline CLIP zero-shot: The original CLIP paper (Radford et al., 2021) reports zero-shot ImageNet top-1 accuracy of 76.2% using ViT-L/14. Our system uses the same ViT-L/14 backbone but augments it with local crop re-ranking and BM25 hybrid fusion, which are not present in vanilla CLIP retrieval. The measured mAP of 0.9177 on our Flickr8k subset demonstrates that the two-stage pipeline preserves CLIP’s strong semantic alignment while adding spatial sensitivity.

vs. Google Photos / Apple Photos: These commercial systems use proprietary models with significantly larger training corpora and GPU-backed infrastructure. Media-Search cannot match their scale or speed; however, it offers two critical differentiators: (i) *complete data privacy* since all processing occurs on the user’s own hardware with no cloud dependency, and (ii) *full transparency* as an open-source system whose retrieval logic can be inspected, modified, and extended.

6.3 Performance Observations

Table 6.4: Search Response Times (500 images, Hardware Accelerated, NVIDIA RTX 3050)

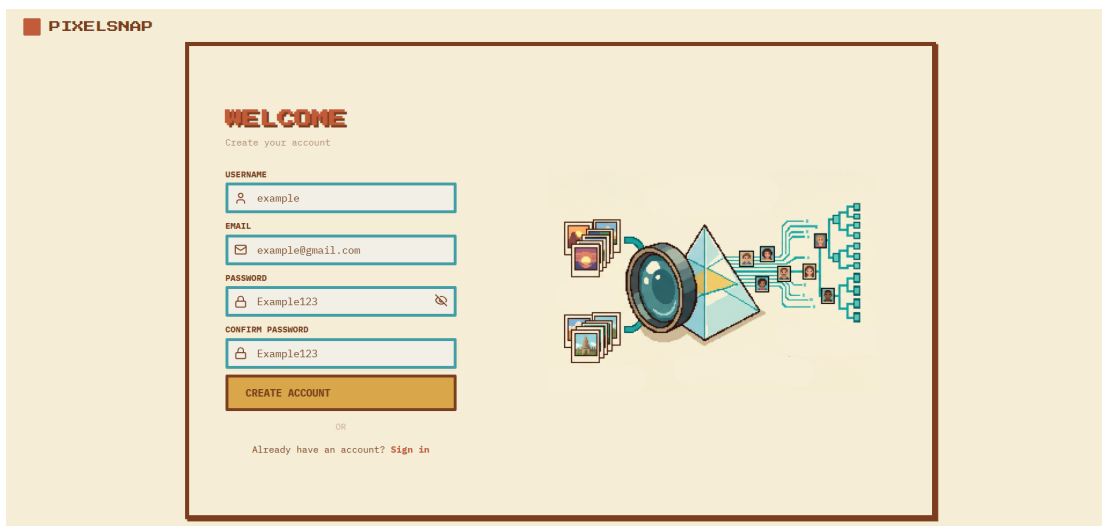
Operation	Min (ms)	Avg (ms)	Max (ms)
CLIP text encoding	45	52	78
Qdrant ANN search (top-20)	2	3	8
Mode 1 – global CLIP (end-to-end)	55	65	95
Mode 2 – re-rank (end-to-end)	210	290	480
Mode 3 – deep hybrid (end-to-end)	180	240	390
Image upload + CLIP embed	310	380	650
VLM description generation	8,200	11,400	18,600
Face detection (1 face per image)	1,100	1,450	2,800

Key observations:

- Mode 1 median response time (65 ms) is well under the 500 ms NFR.
- Mode 2 median (290 ms) is acceptable; it scales with the default candidate set size $k = 20$ (five CLIP image encoder calls each).
- VLM generation (median 11.4 s) runs entirely in the background and has no impact on interactive search latency.
- Face detection time scales linearly with the number of faces detected in each image.

6.4 System Screenshots

The following figures demonstrate the key user interfaces and functionalities of the Media-Search system.



The registration form for PIXELSNAP features a 'WELCOME' heading and a 'Create your account' instruction. It includes input fields for USERNAME (example), EMAIL (example@gmail.com), PASSWORD (Example123), and a CONFIRM PASSWORD field (Example123). A 'CREATE ACCOUNT' button is positioned below these fields. An 'OR' separator and a 'Sign in' link are located at the bottom. To the right of the form is a graphic showing a camera lens, image thumbnails, and a circuit-like data flow.

Figure 6.3: User registration form demonstrating the secure signup process

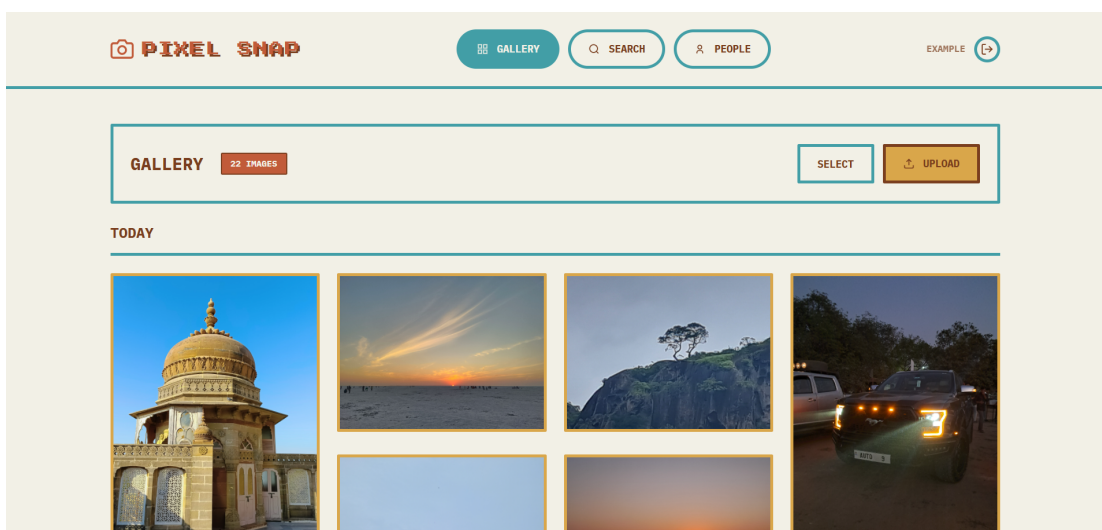


Figure 6.4: Main gallery interface displaying all uploaded media for the authenticated user

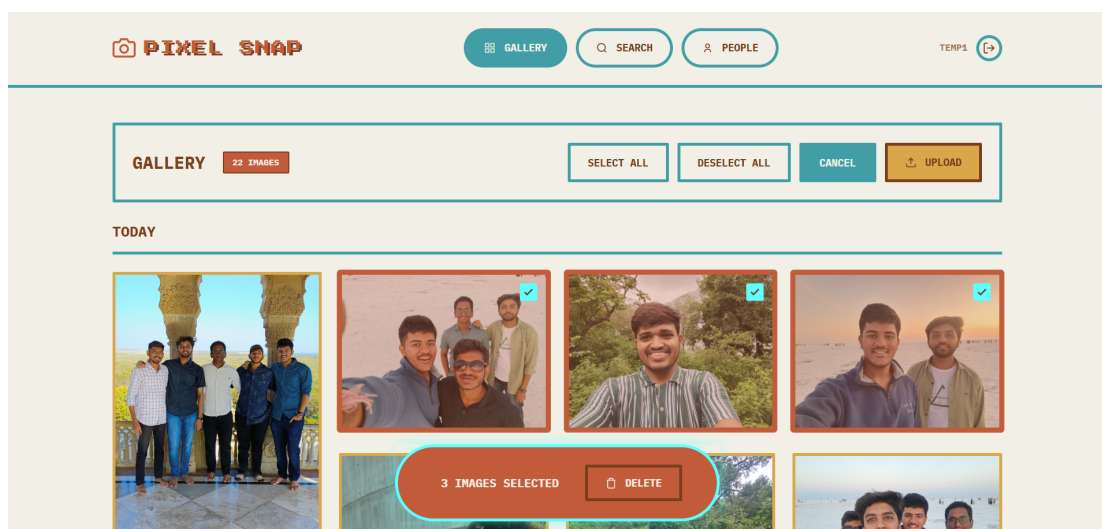


Figure 6.5: Detailed view of a selected image within the gallery

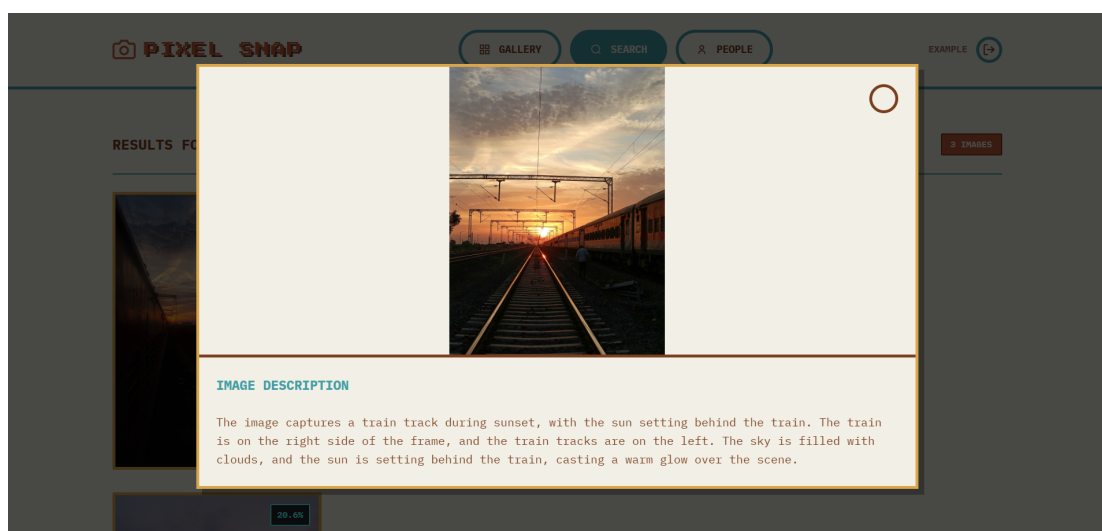


Figure 6.6: AI-generated natural language description of an image produced by the VLM

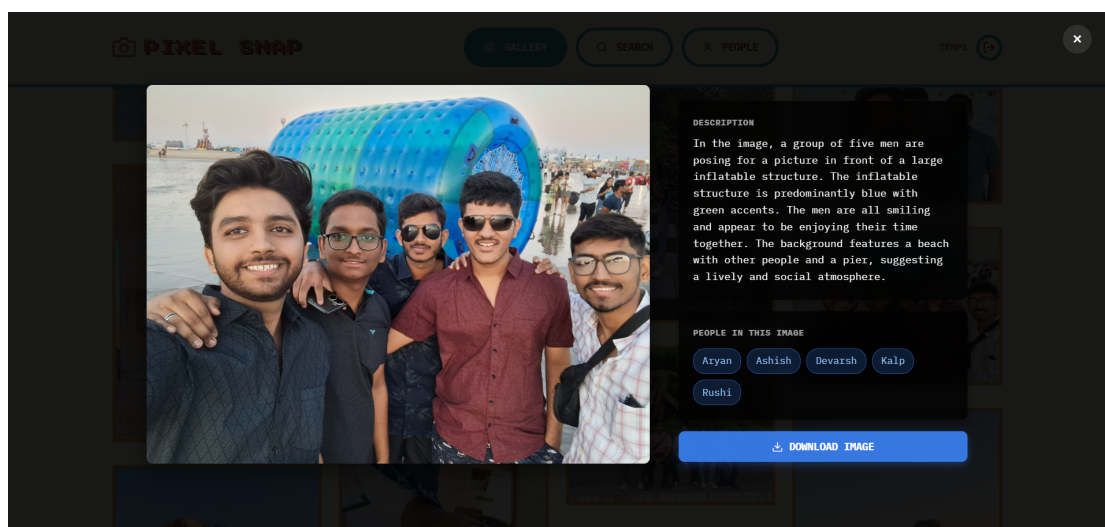


Figure 6.7: Additional example of automated attribute extraction and scene description

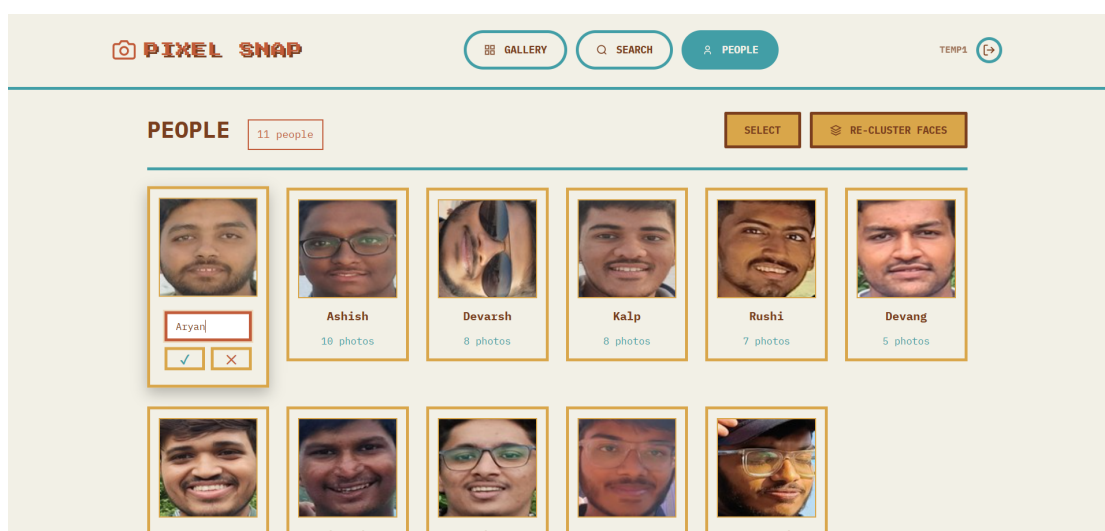


Figure 6.8: Face clusters view grouping detected individuals automatically

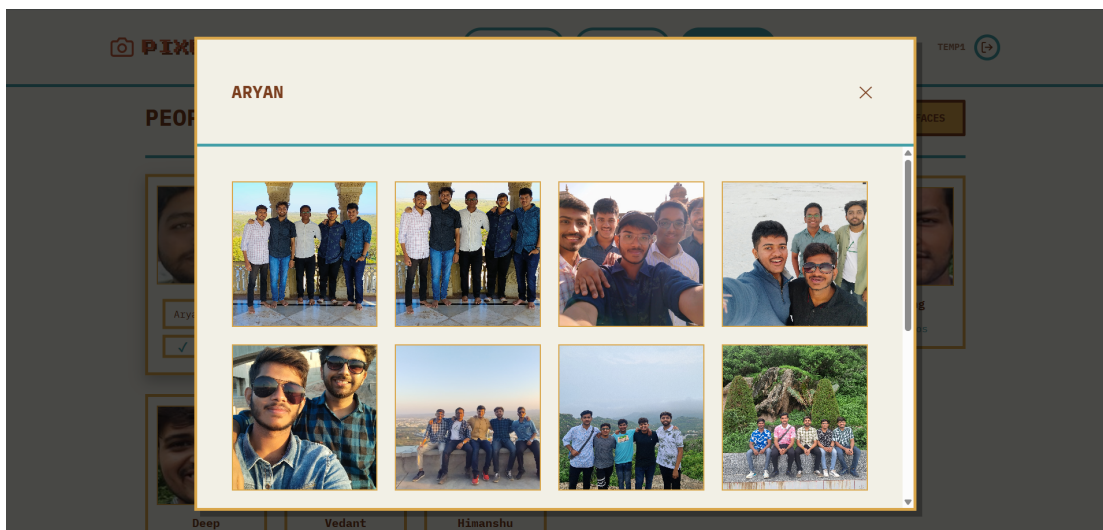


Figure 6.9: Detailed view of all images associated with a specific person

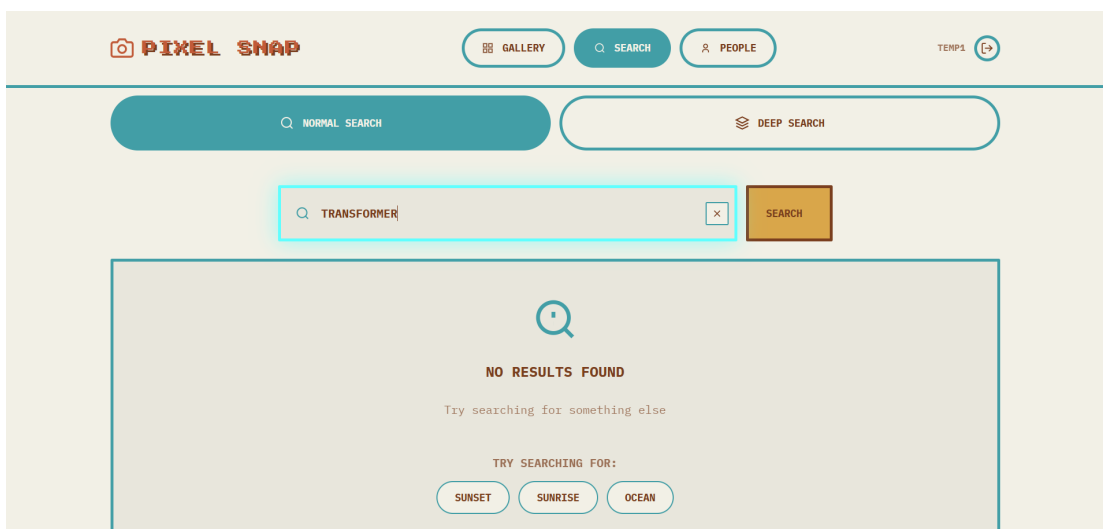


Figure 6.10: Search interface displaying the fallback state when no matching results are found

6.5 Failure Analysis

While the system generally performs exceptionally well for consumer image collections, detailed testing revealed several failure cases that highlight the inherent limitations of the underlying AI models:

1. **CLIP Concept Confusion:** CLIP embeddings occasionally fail to safely distinguish between closely related semantic concepts with similar visual structures. For example, a query for “fox in the snow” erroneously returned high-ranking images of an orange domestic cat. CLIP focuses heavily on broad concept presence rather than fine-grained taxonomy.

2. **Incorrect Face Clustering:** The unsupervised DBSCAN clustering algorithm misclassified distinct individuals as the same person (false positive merges) when lighting conditions or facial angles were severely distorted. It also fractured images of the same individual into separate clusters (false negative splits) if the person appeared across vastly different age ranges (e.g., childhood vs. adulthood).
3. **VLM Hallucinated Descriptions:** The SmolVLM model occasionally fabricated details that were logically plausible but factually absent from the image. For instance, upon analyzing a photograph of a cup of coffee on a wooden desk next to a blank notebook, SmolVLM generated a description stating: "...a cup of coffee next to a notebook with writing on it." These text hallucinations directly degrade the precision of the BM25 Deep Search mode.

Chapter 7

Conclusion

7.1 Summary of Contributions

This project designed, implemented, and deployed a full-stack AI-powered image search platform, **Media-Search**, with the following contributions:

1. **Multi-Modal Semantic Search:** A two-stage CLIP retrieval pipeline (ViT-L/14 global recall + local crop re-ranking) enabling accurate natural language image search without manual labelling.
2. **Hybrid Search:** BM25 over SmolVLM-generated descriptions combined with CLIP semantic scoring, outperforming either signal alone for attribute-specific queries.
3. **Automated Face Organisation:** End-to-end face detection and clustering (DeepFace + DBSCAN) allowing search by person name.
4. **Secure Multi-User Architecture:** JWT authentication, bcrypt hashing, and per-user data isolation (filesystem + Qdrant payload filtering) supporting multiple concurrent users.
5. **Real-Time Feedback:** Server-Sent Events infrastructure streaming live AI processing status to the React frontend.
6. **Open, Self-Hosted Deployment:** A completely open-source system running on commodity hardware with Docker Compose.

7.2 Limitations

- **High VRAM Requirement:** VLM description generation and face detection can be VRAM-intensive. Using CUDA acceleration on an entry-level GPU like the RTX 3050 drastically reduces times, but simultaneous large library uploads must be queued carefully to avoid out-of-memory (OOM) errors.
- **DBSCAN Clustering Errors:** The parameter-free nature of DBSCAN is advantageous, but fixing the hyperparameters (ϵ , `min_samples`) leads to unavoidable clustering errors (splitting or merging identities) when processing large, demographically diverse collections with overlapping facial embeddings.
- **Dependence on Pretrained Models:** The system's intelligence relies strictly on the

pre-existing biases and capabilities of OpenAI's CLIP and HuggingFace's SmolVLM. It cannot dynamically "learn" new concepts that the models did not encounter during their original web-scale training phase.

- Video content is not supported; only static images are indexed.

7.3 Future Work

- **Multi-GPU Scaling:** Distributing inference tasks across multiple GPUs to handle extremely large simultaneous upload streams in a multi-user environment.
- **Video Frame Search:** Keyframe extraction from video files and indexing in Qdrant would enable video content search.
- **Fine-Tuned CLIP:** Domain-specific fine-tuning on medical, satellite, or professional imagery would improve retrieval performance for specialised libraries.
- **Microservices Architecture:** Separating CLIP, VLM, and face services into independent containers with a message queue would enable independent scaling and zero-downtime updates.
- **Active Learning for Clustering:** Incorporating user feedback (merge/split clusters) into periodic DBSCAN re-clustering would improve identity grouping over time.
- **Mobile Client:** A React Native mobile app would allow on-device capture and direct upload from smartphones.

7.4 Concluding Remarks

Media-Search demonstrates that state-of-the-art AI image retrieval, previously confined to resource-rich cloud providers, can be delivered as a self-hosted, privacy-preserving application on a single commodity server. The integration of CLIP, SmolVLM, DeepFace, DBSCAN, and Qdrant within a modular FastAPI backend produces a system that is both technically sophisticated and practically useful. The project provides valuable hands-on experience in vector database technology, asynchronous Python web development, multi-modal AI integration, and modern React frontend engineering, and lays a solid foundation for the future extensions described above.

References

- [1] A. Radford, J. W. Kim, C. Hallacy et al., “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. ICML*, pp. 8748–8763, 2021.
- [2] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., “An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale,” in *Proc. ICLR*, 2021.
- [3] G. Ilharco, M. Wortsman, R. Wightman et al., “OpenCLIP,” *Zenodo*, 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.5143773>
- [4] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid Loss for Language Image Pre-Training,” in *Proc. ICCV*, pp. 11975–11986, 2023.
- [5] HuggingFace, “SmolVLM: Small Vision Language Model,” *HuggingFace Hub*, 2024. [Online]. Available: <https://huggingface.co/HuggingFaceTB/SmolVLM-Instruct>
- [6] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proc. IEEE CVPR*, vol. 1, pp. 511–518, 2001.
- [7] S. Serengil and A. Ozpinar, “DeepFace: A Lightweight Face Recognition and Facial Attribute Analysis Framework for Python,” in *Proc. IEEE INNS EANN*, 2021.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise,” in *Proc. KDD*, pp. 226–231, 1996.
- [9] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Trans. PAMI*, vol. 42, no. 4, pp. 824–836, 2020.
- [10] Qdrant Team, “Qdrant: High-Performance Vector Search Engine,” *GitHub*, 2023. [Online]. Available: <https://github.com/qdrant/qdrant>
- [11] S. E. Robertson and S. Walker, “Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval,” in *Proc. ACM SIGIR*, pp. 232–241, 1994.
- [12] V. Karpukhin, B. Oguz, S. Min et al., “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, pp. 6769–6781, 2020.

List of Abbreviations

AI	Artificial Intelligence
ANN	Approximate Nearest Neighbour
API	Application Programming Interface
BM25	Best Match 25
CBIR	Content-Based Image Retrieval
CLIP	Contrastive Language-Image Pre-training
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CUDA	Compute Unified Device Architecture
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DFD	Data Flow Diagram
ER	Entity-Relationship
FR	Functional Requirement
GPU	Graphics Processing Unit
GTU	Gujarat Technological University
HNSW	Hierarchical Navigable Small World
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
ML	Machine Learning
NFR	Non-Functional Requirement
ORM	Object-Relational Mapper
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
SSE	Server-Sent Events
TF-IDF	Term Frequency – Inverse Document Frequency
UUID	Universally Unique Identifier
VGEC	Vishwakarma Government Engineering College
ViT	Vision Transformer
VLM	Vision-Language Model